

SPRAWOZDANIE NR. 2

Małgorzata Pawlisiak 163776
Bioinformatyka semestr III

Opis zastosowanego formatu zapisu grafu:

- Do zapisu pliku wejściowego(zawierającego dowolny graf skierowany) i wyjściowego(zawierającego graf oryginalny H) wykorzystano plik o formacie .txt, Format został zaprojektowany tak, aby łatwo wczytywać listę wierzchołków i listę krawędzi oraz by nie pomijać wierzchołków izolowanych.
- Pierwszą linią jest lista wszystkich wierzchołków grafu(nazwą wierzchołków odpowiadają wielkie litery alfabetu). Kolejne linie pliku to lista sąsiedztwa zawierająca kolejno wierzchołek początkowy krawędzi, wierzchołek końcowy krawędzi oraz etykietę wierzchołka w formacie: e0, e1, ..., en.

Przykładowy format zapisu grafu zastosowany w zadaniu:

```
A B C D E F G H I J K L → Lista wierzchołków
A C e1
B C e2
C E e3 → Lista sąsiedztwa
D E e4
E F e5
F G e6 → Etykieta krawędzi
G H e7
H I e8
I J e9
J K e10
K L e11
```

Struktura programu:

- Klasa EdgeG i EdgeH:** Opisują każdą krawędź grafu G lub H, zawierają pole name, które jest etykietą krawędzi w przypadku **EdgeG**: zapisaną w formacie: **e0, e1, e2** itd., a w przypadku **EdgeH**: **nazwą wierzchołka**, ponieważ po transformacji grafu sprzężonego w graf oryginalny wierzchołki stają się krawędziami.

Każda klasa opisująca wierzchołek ma także pola określające, które wierzchołki łączy krawędź: u to poprzednik, a v to następnik.

- Klasa Vertex:** Opisuje każdy wierzchołek grafu G, zawiera pole name, które jest etykietą wierzchołka zapisaną wielkimi literami alfabetu. Każdy obiekt klasy Vertex posiada także wektor successors, który przechowuje następniki każdego wierzchołka, czyli ma w sobie liste indeksów do innych obiektów klasy. Dzieje się to za pomocą wektora vertices, który jest zadeklarowany w funkcji menu.

- **Funkcja Menu:**
 - Pobiera od użytkownika nazwę pliku wejściowego.
 - Tworzy wektory vertices (wierzchołki G) i edgeG (krawędzie G).
 - Wywołuje funkcję **ReadGraph** – wczytuje graf G - jeśli nie można wczytać pliku lub występuje błąd formatu, zwraca błąd i przerywa działanie programu.
 - Wyświetla liczbę wczytanych wierzchołków i krawędzi.
 - Wywołuje funkcję **AdjointGraph** - sprawdza, czy graf G jest grafem sprzężonym. Wypisuje informację, czy graf jest sprzężony, czy nie.
 - Tworzy wektory edgeH(krawędzie H) i indexingGtoH(lista reprezentantów krawędzi grafu H - wierzchołków grafu G).
 - Wywołuje funkcję **ReconstructGraph**(tworzy graf H).
 - Wywołuje funkcję **LineGraph**(sprawdza liniowość). Wypisuje informację, czy graf jest liniowy, czy nie.
 - Wywołuje funkcję **SaveGraph** - zapisuje graf H do pliku w formacie pliku wejściowego, tak aby później można było podać wyjście na wejście programu.
 - Zapisuje graf H do pliku rekonstrukcja_nazwa pliku wejściowego.txt.
- **Wczytanie grafu G - Funkcja ReadGraph:**
 - Czyta pierwszą linię pliku (lista nazw wierzchołków) i tworzy obiekty Vertex zapisując ich nazwę w zmiennej name, a następnie wszystkie utworzone obiekty umieszcza w wektorze vertices za pomocą polecenia push_back.
 - Czyta kolejne linie pliku, które zawierają opisy krawędzi: wczytywane są wierzchołki początkowe i końcowe krawędzi jako zmienne u i v oraz name(etykieta krawędzi).
 - Dla każdej krawędzi odnajdujemy indeksy ia i ib odpowiadające literom z pierwszej linii i dodajemy krawędź do wektora edgeG oraz uzupełniamy wektor successors(listę następników wierzchołka) dla każdego wierzchołka. Otrzymujemy dzięki temu kompletną listę następników, którą wykorzystamy w sprawdzeniu czy graf jest sprzężony.
 - Funkcja sprawdza poprawność, zwracając false w przypadku błędów.
- **Sprawdzenie, czy graf G jest sprzężony - Funkcja AdjointGraph:**

$N^+(x) \cap N^+(y) \neq \emptyset \Rightarrow N^+(x) = N^+(y) \rightarrow$ Warunek sprzężenia grafu

Implementacja:

 - Dwie zagnieżdżone pętle, które przechodzą po wszystkich parach wierzchołków.
 - Dla każdej pary porównujemy ich wektory successors(porównujemy ich następniki): jeśli znajdziemy wspólny następnik, ustawiamy common=true.
 - Graf jest sprzężony jeśli wszystkie następniki wierzchołków są rozłączne lub jeśli są identyczne.
 - Gdy common==true, to sprawdzamy czy lista następników obu wierzchołków jest identyczna, jeśli są one identyczne to graf jest sprzężony jeśli nie to nie jest.
- **Rekonstrukcja grafu oryginalnego H - Funckje ReconstructGraph + CompressReps:**
 - Każdy wierzchołek grafu G zastępujemy w grafie H rozłącznym łukiem (start, end). Następnie dla każdej krawędzi $u \rightarrow v$ w G łączy koniec łuku u z początkiem łuku v w grafie H poprzez scalenie odpowiednich indeksów.

- Tworzy wektor reps, o wielkości odpowiadającej podwojonej ilości wierzchołków grafu G, gdzie każdy indeks przyjmuje wartość od 0 do $2 \times \text{ilość wierzchołków}$ (każdy indeks symbolizuje osobny punkt start/end).
- Tworzy obiekty klasy edgeH i dodaje je do wektora – dla każdego wierzchołka grafu G dodaje krawędź $(2 \cdot i, 2 \cdot i + 1)$ - rozłącze łuki i jako name przypisuje nazwę wierzchołka z G, z którego powstał łuk.
- Dla każdej krawędzi $\text{edgeG}[k] = (u, v)$ tworzy zmienne $u_end = 2 \cdot u + 1$ (koniec łuku odpowiadającego u) i $v_start = 2 \cdot v$ (początek łuku odpowiadającego v).
- Łączy końce łuku z początkami łuków po przez przypisanie do nowej zmiennej keep u_end i v_start do nowej zmiennej replace. Nazwy zmiennych odnoszą się do faktu, że zamieniamy indeks początku kolejnego łuku na indeks końca poprzedniego łuku.
- Wywołuje funkcję pomocniczą CompressReps, która edytuje wektor reps (łączy rozłączne łuki) po przez zamianę replace na keep.
- Aktualizuje wszystkie wystąpienia replace na keep w wektorze edgeH.
- Po przejściu wszystkich krawędzi otrzymujemy reps z powtarzającymi się wartościami – to oznacza, które pierwotne punkty zostały skompresowane razem.
- Tworzy zmienną $\text{maxRep} = \max(\text{reps}[i])$ - największy reprezentant w reps.
- Tworzy wektor newLabel o rozmiarze $\text{maxRep} + 1$ z początkowymi wartościami -1. Iterując po reps nadje kolejno nowe etykiety reprezentantom: pierwszy napotkany reprezentant dostaje 0, następny 1, itd.
- Dla każdej krawędzi w edgeH zamieniamy jej stare indeksy końców na spójne, ponumerowane od nowa etykiety, korzystając z tego, że reps mówi, który reprezentant należy do którego wężła, a newLabel przypisuje każdemu reprezentantowi nowy numer.
- Dla każdego punktu dawnego łuku zapisujemy w wektorze indexingGtoH jego ostateczny numer w grafie H, czyli przypisujemy mu etykietę nadaną reprezentantowi przez newLabel.
- Przykład w jaki sposób funkcja tworzy rozłączne łuki, a potem je scala tworząc graf H na przykładzie bardzo prostego grafu A B C:

A->B

A->C

B->C

Początkowe, rozłączne łuki:

A -> {0,1}

B -> {2,3}

C -> {4,5}

reps = [0,1,2,3,4,5]

edgeH = {{0,1},{2,3},{4,5}}

Po kompresji:

reps = [0,1,1,3,1,5]

Utworzenie newLabel i przypisanie etykiet reprezentantom:

newLabel = [-1,-1,-1,-1,-1,-1]

newLabel = [0,1,-1,2,-1,3]

Przypisanie każdej krawędzi nowych indeksów:

edgeH[0] = {0,1} -> A

edgeH[1] = {1,2} -> B

$\text{edgeH}[2] = \{1, 3\} \rightarrow C$

$\text{edgeH} = \{ \{0, 1\}, \{1, 2\}, \{1, 3\} \}$

- **Sprawdzenie czy graf H jest liniowy - LineGraph:**
 - Grafem liniowym jest graf sprzężony, który jest 1-grafem, czyli nie ma krawędzi wielokrotnych. Co za tym idzie graf liniowy nie może zawierać jako podgraf żadnej z podanych niżej struktur:
 - Warunek liniowości sprawdzamy, porównując każdą parę krawędzi grafu H: jeśli znajdziemy dwie krawędzie o tym samym początku i tym samym końcu, to oznacza krawędź wielokrotną, czyli graf nie jest liniowy - wtedy przerywamy działanie funkcji i zgłaszamy błąd.
- **Zapisanie grafu H - SaveGraph:**
 - Funkcja zapisuje plik w tym samym formacie, co plik wejściowy:
Pierwsza linia: $e_0 e_1 e_2 \dots e_{(k-1)}$ gdzie k to liczba wierzchołków H.
Lista krawędzi: każda linia $eU eV e$ (etykieta - nazwa wierzchołka grafu G)
 - Funkcja najpierw oblicza maxLabel (największy indeks w indexingGtoH i edgeH), ustala $\text{count} = \text{maxLabel} + 1$, zapisuje listę wierzchołków, a następnie listę krawędzi.

Oszacowanie złożoności algorytmu:

ReadGraph:

- Wczytanie wierzchołków: $O(|V(G)|)$
- Wczytanie krawędzi: $O(|E(G)|)$
- Dla każdej krawędzi przeszukujemy listę wierzchołków $O(|E(G)| \times |V(G)|)$

AdjointGraph:

- Przechodzimy po wszystkich parach wierzchołków: $O(|V(G)|^2)$
- Dla każdej pary porównujemy listy następców; w najgorszym przypadku mogą mieć długość $O(|V(G)|)$, więc złożoność rośnie do: $O(|V(G)|^3)$

ReconstructGraph:

- Utworzenie początkowego reps i edgeH : $O(|V(G)|)$
- Kompresja rozłącznych łuków (CompressReps), wykonywana dla każdej krawędzi G: każda operacja kosztuje $O(|V(G)|)$, więc $O(|E(G)| \cdot |V(G)|)$
- Aktualizacja edgeH po scaleniu: $O(|E(G)| \cdot |V(G)|)$
- Tworzenie newLabel oraz mapowanie indeksów: $O(|V(G)| + |E(G)|)$

LineGraph:

- Sprawdzanie każdej pary krawędzi w H: $O(|E(H)|^2)$

SaveGraph:

- wypisanie wszystkich wierzchołków $H \rightarrow O(|V(H)|)$
- wypisanie wszystkich krawędzi $H \rightarrow O(|E(H)|)$

Całkowita złożoność algorytmu: $O(|V(G)|^3)$

Największy koszt obliczeniowy w programie generuje funkcja `AdjointGraph` o złożoności $O(|V(G)|^3)$, podczas gdy pozostałe funkcje, choć mogą być kosztowne dla dużych grafów (`ReconstructGraph` $O(|E(G)| \cdot |V(G)|)$, `LineGraph` $O(|V(G)|^2)$), mają mniejszy wpływ na czas działania programu.

Opis przeprowadzonych testów:

Program został przetestowany na różnych klasach grafów wejściowych, obejmujących grafy z wierzchołkami izolowanymi, krawędziami wieloliterowymi oraz cykliczne.

ReadGraph: sprawdzono poprawność wczytywania wierzchołków i krawędzi, tworzenie listy następców oraz wykrywanie błędów w pliku wejściowym.

AdjointGraph: testowano wykrywanie grafów sprzężonych i niesprzężonych.

ReconstructGraph + CompressReps: weryfikowano prawidłowe łączenie rozłącznych łuków, generowanie reps i newLabel oraz zgodność grafu H z oczekiwaniami.

LineGraph: sprawdzano wykrywanie krawędzi wielokrotnych oraz potwierdzanie liniowości grafu.

SaveGraph: testowano zapis pliku w poprawnym formacie i możliwość ponownego wczytania grafu H.

Wszystkie testy potwierdziły poprawność działania programu: graf H jest prawidłowo rekonstruowany, sprzężenie i liniowość wykrywane poprawnie, a zapis i obsługa błędów działają stabilnie i zgodnie ze specyfikacją.